

Databricks Questions & Answers

Compiled from the questions visible in this chat (Questions 36-48). Each entry includes the question, options, and selected answer.

Question 36

A Unity Catalog table `customer_data` in schema `finance` contains sensitive fields such as `ssn` and `credit_score`. Intern Group should only see masked values, while Analyst Group should only access rows for their assigned region. The engineer needs to restrict access based on user role and region without duplicating data. How should this security policy be enforced?

- A. Create views using `current_user()` and `is_account_group_member()` functions, and apply masking logic inside the SQL `SELECT` clause for each sensitive column.
- B. Use Unity Catalog row filters based on the region and column masks based on user roles.
- C. Create dynamic views for each user role and manage access with ACLs.
- D. Use Unity Catalog row filters based on user roles and column masks based on the region.

Correct answer: B - Use Unity Catalog row filters based on region and column masks based on user roles.

Question 37

A production Lakeflow Spark Declarative Pipeline processes customer transaction data and has SQL `EXPECT` data-quality expectations such as `transaction_amount > 0` and `customer_id IS NOT NULL`. For the most recent pipeline update, the engineer wants to programmatically retrieve each expectation name, associated dataset, and counts of records that passed and failed from the `event_log_table`. Which method retrieves the desired metrics?

- A. Access `event_log_table`, filter for events where `event_type = 'flow_progress'`, and parse `details.flow_progress.data_quality.expectations`.
- B. Use the Lakeflow Spark Declarative Pipelines UI Data Quality tab and manually retrieve the metrics.
- C. Query `event_log_table` for `event_type = 'data_quality'` and directly select `passed_records` and `failed_records` fields.
- D. Filter `event_log_table` for `event_type = 'expectation_result'` and extract expectation metrics from details.

Correct answer: A - Filter flow_progress events and parse details.flow_progress.data_quality.expectations.

Question 38

A task-management system processes task events in near real time with Lakeflow Spark Declarative Pipelines. A streaming table `tasks_status` stores the latest status of each task and has deletion vectors, row tracking, and change data feed enabled. A new pipeline must enrich `tasks_status` in near real time with the task owner's department from a static employee dimension table. How should this be implemented?

- A. Read the `tasks_status` change data feed with `readStream()` and `readChangeFeed`, enrich with employee, write to a new streaming table, and use `apply_changes()` to process the enriched changes.
- B. Use `readStream()` with `skipChangeCommits` to read `tasks_status`, enrich with employee, and store the result in a new streaming table.
- C. Use `read()` to read `tasks_status`, enrich with employee, and store the result in a materialized view.
- D. Use `readStream()` directly on `tasks_status`, enrich with employee, and store the result in a new streaming table.

Correct answer: A - Consume the CDF, enrich it, and apply the changes to the target streaming table.

Question 39

A platform engineer needs to report resource consumption categorized by SKU tier across all workspaces using system.billing.usage. Which SQL query accurately returns daily usage by product?

- A. `SELECT date_trunc('day', usage_start_time) AS usage_day, sku_name, SUM(usage_quantity) AS total_daily_dbus FROM system.billing.usage WHERE usage_quantity IS NOT NULL GROUP BY usage_day, sku_name ORDER BY usage_day;`
- B. `SELECT TO_DATE(usage_start_time) AS usage_day, sku_name, SUM(usage_quantity) AS total_dbus FROM system.billing.usage GROUP BY usage_day ORDER BY usage_day;`
- C. `SELECT CAST(usage_start_time AS DATE) AS usage_day, sku_name, COUNT(usage_quantity) AS dbu_count FROM system.billing.usage GROUP BY usage_day, sku_name ORDER BY usage_day;`
- D. `SELECT DATE_TRUNC('day', usage_start_time) AS day, sku_name, SUM(dbus) AS daily_dbus FROM system.billing.usage WHERE sku_name IS NOT NULL GROUP BY day, sku_name;`

Correct answer: A - Group by day and sku_name and SUM(usage_quantity).

Question 40

In a Databricks Asset Bundle project, resources/app.yml should deploy a Databricks App named databricks_app_deployed and a Volume named volume_deployed, and grant the service principal behind the Databricks App READ and WRITE permissions on the Volume. Which configuration achieves this?

- A. Uses the app resource id itself as the Volume grant principal.
- B. Uses the Databricks App service principal client ID as the Volume grant principal, with READ_VOLUME and WRITE_VOLUME privileges.
- C. Uses an app-name/id expression as the Volume grant principal.
- D. Uses another app-name/id expression rather than the service principal client ID.

Correct answer: B - Use \${resources.apps.databricks_app_deployed.service_principal_client_id} as principal, with READ_VOLUME and WRITE_VOLUME. Note: during the chat this was briefly labeled C due to the photographed option layout; the technically correct configuration is the one using service_principal_client_id.

Question 41

A large retail sales DataFrame has salesperson_id, region, sale_amount, and sale_date. The engineer must rank salespeople within each region by total cumulative sales, highest rank = 1, with equal totals sharing the same rank, using a PySpark window and dense_rank(). Which snippet is correct?

- A. Aggregate by salesperson_id and region, partition by region/order by total_sales descending, but use F.rank().
- B. Aggregate by salesperson_id and region; window = Window.partitionBy('region').orderBy(F.desc('total_sales')); then add F.dense_rank().over(window).
- C. Aggregate by salesperson_id and region and use dense_rank(), but create a global window without partitionBy('region').
- D. Do not aggregate; partition raw sales by region, order by individual sale_amount, and apply dense_rank().

Correct answer: B - Aggregate total sales first, partition by region, order total_sales descending, then use dense_rank().

Question 42

A data engineer ingests JSON files from cloud object storage using Databricks Auto Loader. Large files may occasionally arrive. The team wants predictable micro-batch sizes by throttling the volume of data scanned at 1 GB regardless of file count. Which Auto Loader configuration should be used?

- A. Configure `cloudFiles.maxBytesPerTrigger` with 1GB.
- B. Configure `cloudFiles.maxPartitionBytes` with 1GB.
- C. Configure `cloudFiles.maxFilesPerTrigger` and estimate average file size to approximate 1GB.
- D. Configure `cloudFiles.maxSizePerTrigger` with 1GB.

Correct answer: A - `cloudFiles.maxBytesPerTrigger` = 1GB.

Question 43

A `groupBy` aggregation on a massive user activity log is grouped by `user_id`. A few users have millions of records, causing task skew and long runtimes. Which technique fixes the skew?

- A. Use `reduceByKey` instead of `groupBy` to avoid shuffles.
- B. Filter out the skewed users before aggregation.
- C. Use salting by adding a random prefix to skewed keys before aggregation, then aggregate again after removing the prefix.
- D. Increase Spark driver memory and retry.

Correct answer: C - Use key salting and a two-stage aggregation.

Question 44

Which approach demonstrates a modular and testable way to use `DataFrame.transform` for ETL code in PySpark?

- A. Put transformation logic in a Pipeline class method and call the method directly in the test.
- B. Define `upper_transform(df)`, call `actual = test_input.transform(upper_transform)`, and compare actual with expected.
- C. Define separate `upper_value` and `filter_positive` functions and chain `df.transform(upper_value).transform(filter_positive)`, without showing the focused test pattern asked for.
- D. Define a monolithic `transform_data(input_df)` function and test it directly rather than using `DataFrame.transform`.

Correct answer: B - Define a reusable transformation function and pass it to `DataFrame.transform()`, then unit-test the result.

Question 45

A company ingests semi-structured JSON with Auto Loader. Some records contain null critical fields, invalid types, or unexpected nested schema variations. Bad records must not be silently dropped; they must be routed to a quarantine Delta table while good records continue into Bronze without failing the job, supporting batch and streaming. Which approach fulfills the quarantine mechanism?

- A. Use `inferSchema=True` and `foreachBatch()` with `try/except` to redirect failed batches.
- B. Use Auto Loader with `failFast=false` and schema evolution, allowing invalid records to be silently ignored.
- C. Use Lakeflow Spark Declarative Pipelines and implement an `EXPECT()` constraint with record-audit logic to route bad records.
- D. Drop rows with null critical fields and rely on audit logs for malformed data.

Correct answer: C - Use Lakeflow Spark Declarative Pipelines expectations with audit/quarantine routing logic.

Question 46

A data engineer must productionize a Spark application with numerous external library dependencies and custom environment variables and Spark configuration parameters. Which TWO methods help accomplish this?

- A. Use secrets in init scripts to store configuration data.
- B. Use compute policies to set system properties, environment variables, and Spark configuration parameters.
- C. Add libraries to compute policies.
- D. Create init scripts on DBFS.
- E. Install libraries on DBFS.

Correct answer: B and C - Use compute policies for configuration/environment settings and library policy controls.

Question 47

An existing production Databricks job is being brought under Asset Bundle management. Its current configuration should be captured as YAML with referenced files included, and future bundle deployments must update the existing job in place rather than create a new job. How should this be done?

- A. Run bundle generate job --existing-job-id and deploy; deployment will always update the existing job automatically without binding.
- B. Export the job as JSON, manually convert it to YAML, and deploy.
- C. Manually recreate the YAML to match the existing job and deploy.
- D. Run databricks bundle generate job --existing-job-id to generate YAML/download referenced files, then bind the bundle job resource to the existing Databricks job before deployment.

Correct answer: D - Generate the job configuration, then bind the bundle resource to the existing job so deployments update it in place.

Question 48

A Delta table must continuously retain deleted files for 15 days instead of the default 7 days to comply with a retention policy. Which code snippet correctly sets this retention period?

- A. `spark.sql("ALTER TABLE my_table SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' = 'interval 15 days')")`
- B. Set a `deletedFileRetentionDuration` attribute directly on a `DeltaTable` Python object.
- C. `spark.sql("VACUUM my_table RETAIN 15 HOURS")`
- D. `spark.conf.set('spark.databricks.delta.deletedFileRetentionDuration', '15 days')`

Correct answer: A - Set the table property `delta.deletedFileRetentionDuration` to 'interval 15 days'.